OBJECT-ORIENTED PROGRAMMING (JAVA) · OVERVIEW

Arrays, 2D/3D, and ArrayLists

Overview and objectives

- **Course:** Intermediate Object-Oriented Programming (Java)
- **Texts:** *Core Java, Volume I* (13th ed.) and *Core Java, Volume II* (13th ed.)
- **Time:** about 3.75 hours across three class days
- **Prerequisite you already have:** the basics of one-dimensional arrays

1. The short version

We start where you already are, one-dimensional arrays, and push outward in two directions. First, into **more dimensions**: 2D arrays (grids), ragged arrays, and 3D arrays (stacks of grids). Then, into **flexibility**: the `ArrayList`, which is what you reach for when you do not know how many things you will have until the program runs.

Almost all of our class time will be spent reading and running code together. The slides exist to give you a picture in your head; the real learning happens when you predict what a snippet will print and then we run it to find out.

2. What we will cover

- **Review of 1-D arrays.** Declaration, initialization, length, indexing, the enhanced `for` loop, and the fact that an array variable holds a *reference*, not the data itself.
- **2-D arrays.** How Java really models them ("arrays of arrays"), row-major iteration, `array.length` versus `array[i].length`, and `Arrays.deepToString`.
- **Ragged (jagged) arrays.** Rows that do not all have the same length, and why Java lets you do that.
- **3-D arrays.** Stacking grids into layers, and where that actually shows up.
- **The `Arrays` utility class.** `sort`, `fill`, `copyOf`, `equals`, `binarySearch`, `toString` and `deepToString`.
- **`ArrayList`.** Generics, `add`, `get`, `set`, `remove`, `size`, iterating, autoboxing, size versus capacity, and the handful of traps that bite everyone once.
- **Choosing between an array and an `ArrayList`.** The actual decision you will make on the job.

3. Objectives: by the end, you should be able to

1. **Declare, allocate, and initialize** 1-D, 2-D, and 3-D arrays, and explain the difference between declaring a variable and allocating the storage it points to.

2. **Traverse** any of the above with both counted `for` loops and enhanced `for` loops, and know when only one of the two will do.
3. **Explain** why a Java 2-D array is an array of arrays, and use that model to predict the behavior of ragged arrays and reference sharing.
4. **Use the `Arrays` class** to sort, copy, compare, fill, search, and print arrays instead of writing that code by hand.
5. **Create and manipulate an `ArrayList`**, choose the right iteration idiom, and describe how autoboxing lets it store numbers.
6. **Distinguish size from capacity**, and explain in your own words why an `ArrayList` can grow while an array cannot.
7. **Justify a choice** between an array and an `ArrayList` for a given problem.

4. How to show up ready

Read the arrays material in *Volume I* and the collections material for `ArrayList` before we meet; skim, do not memorize. Come with the syntax roughly in hand so we can spend class time on the *why* and the *gotchas* rather than the spelling. Bring a laptop if you have one; we will be running code live and I will want you to run it too.

I do not grade attendance, quizzes, or busywork. The way you do well here is simple: read ahead, practice the examples on your own, build the project, and demo it to me. That is the whole deal.